NATIONAL UNIVERSITY OF LESOTHO

Department of Mathematics and Computer Science

CS4433 – SOFTWARE ENGINEERING

PHASE 3: SYSTEM DESIGN

Moderated Learning Management System

202322795 K. MOTHOSELE

Software Design Document
Individual Assignment
October 2026

Contents

- 1. Introduction and design scope
- 2. Architecture
- 3. Design-level class model
- 4. Behavioural models
- 5. Database design
- 6. Requirements traceability
- 7. Design decisions and open points
- References

This document addresses Phase 3 of the CS4433 assignment: architecture, detailed class design, behavioural models and database design. It follows the Moderated Learning Management System case study. Where the brief leaves details open, the design decision is identified for confirmation.

1. Introduction and design scope

The system will manage courses, assignments, submissions, marking, moderation and release of assessment results for a secondary school. It will be a C++17 console or simple desktop application using SQLite on a school computer or local network. Web/mobile interfaces, online payment, video conferencing, automated grading and plagiarism detection are out of scope.

1.1 Design objectives

- Enforce role-based access for Student, Lecturer, Head of Department (HoD) and Administrator.
- Prevent students from seeing marks before HoD approval.
- Support individual deadline extensions and resubmission before the applicable deadline.
- Apply the late-work rule consistently and retain written feedback.
- Record every mark change with actor, time, old/new values and reason.
- Keep SQLite access behind a repository/DAO layer and use parameterised queries.

1.2 Business rules

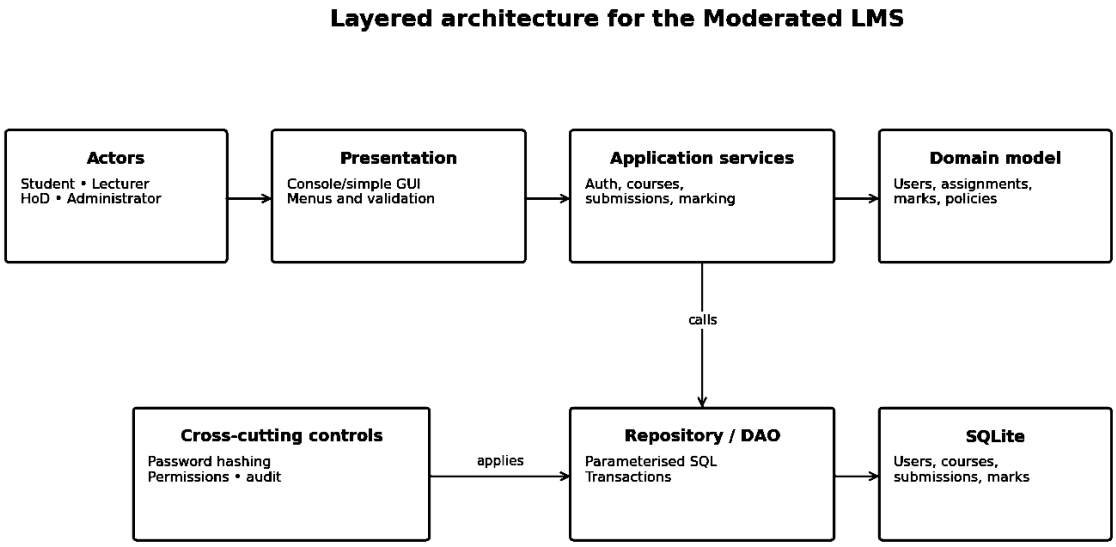
ID	Rule
BR-01	Students access only courses in which they are enrolled.
BR-02	A student may submit or resubmit before the applicable deadline.
BR-03	The applicable deadline is the assignment deadline or a valid individual extension.
BR-04	Late work loses 10% of the available mark per day for up to three days; after that it receives zero.
BR-05	Lecturers record a score and written feedback; marks remain hidden pending HoD approval.
BR-06	The HoD approves a mark or returns it to the lecturer with a reason.
BR-07	Only approved marks can be released to students.
BR-08	Every mark change creates an audit entry: actor, timestamp, old score, new score and reason.

Interpretation to confirm: this design treats each started 24-hour period after the applicable deadline as one late day, capped at three penalty days. The final SRS should specify whether the school instead intends calendar days.

2. Architecture

2.1 Selected style: layered architecture

A layered architecture is selected because the application has a clear separation between user interaction, use-case coordination, domain rules and SQLite persistence. The presentation layer handles menus and input/output; application services coordinate use cases; domain classes hold state and rules; repositories isolate SQL and database transactions.



Component view of the proposed layered architecture.

2.2 Component responsibilities

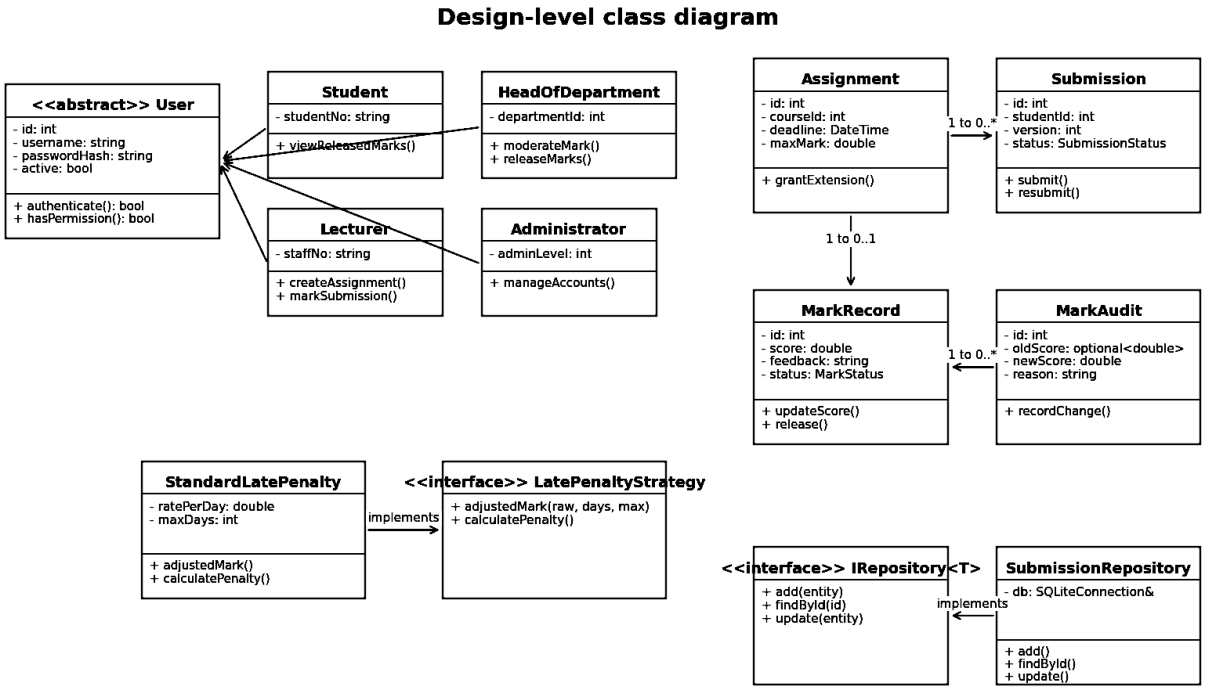
Component	Responsibility	Boundary
Presentation	Role-aware menus, prompts, input validation and feedback.	Does not execute SQL or decide mark-release rules.
Application services	Login, enrolment, submissions, marking, moderation and release.	Coordinates use cases and transactions.
Domain model	Users, assignments, submissions, marks and policy abstractions.	Enforces valid states and business rules.
Repository / DAO	SQL, parameter binding, row mapping and persistence.	Only layer that accesses SQLite directly.
Security and audit	Password hash verification, permission checks and audit events.	Applied to protected actions.
SQLite	Persistent users, courses, enrolments, assignments, submissions and marks.	Foreign keys and constraints preserve integrity.

2.3 Rationale and alternative

The layered approach allows business rules to be tested without a live database and makes persistence changes less disruptive. It also provides a single place to enforce parameterised SQL and transaction handling. Model–View–Controller was considered but rejected: a console or simple desktop interface does not need multiple independently updating views, so MVC would add structure without improving the key separation between business rules and persistence.

3. Design-level class model

The diagram shows principal classes, visibility, types, inheritance and key relationships. Method signatures are representative; implementation should keep the same responsibilities and access restrictions.



Design-level class diagram showing role inheritance, domain classes and pattern interfaces.

Class / interface	Important fields	Operations / responsibility
User (abstract)	- id:int; - username:string; - passwordHash:string; - fullName:string; - active:bool	+ authenticate():bool; + hasPermission(action):bool. Common identity and access contract.
Student : User	- studentNo:string	+ viewReleasedMarks(). Access only released marks for enrolled courses.
Lecturer : User	- staffNo:string	+ createAssignment(); + markSubmission(id, score, feedback).
HeadOfDepartment : User	- departmentId:int	+ moderateMark(markId, decision, reason); + releaseMarks().
Administrator : User	- adminLevel:int	+ manageAccounts(); + manageDepartments(); + manageCourses(); + enrolStudent().
Assignment	- id:int; - courseId:int; - title:string; - deadline:DateTime; - maxMark:double	+ setDeadline(); + grantExtension(studentId, deadline, reason).
Submission	- id:int; - assignmentId:int; - studentId:int; - submittedAt:DateTime; - status:SubmissionStatus; - version:int	+ submit(); + resubmit(); + transitionTo(status).
MarkRecord	- id:int; - submissionId:int; - score:double; - feedback:string; - status:MarkStatus; - releasedAt:optional<DateTime>	+ updateScore(); + submitForModeration(); + release().
MarkAudit	- id:int; - markId:int; - changedBy:int; - changedAt:DateTime; - oldScore:optional<double>; - newScore:double; - reason:string	+ recordChange(). Append-only mark history.
LatePenaltyStrategy (interface)	No required fields	+ calculatePenalty(lateDays:int, maxMark:double):double; + adjustedMark(rawMark, lateDays, maxMark):double.
StandardLatePenalty	- ratePerDay:double=0.10; - maxPenaltyDays:int=3	+ calculatePenalty(); + adjustedMark(). Implements the stated late policy.
IRepository<T> (interface)	Generic entity contract	+ add(entity); + findById(id); + update(entity).
SubmissionRepository	- db:SQLiteConnection&	+ add(); + findById(); + update(); + findByAssignmentAndStudent().

3.1 Relationships and multiplicities

Relationship	Multiplicity / rule
User inheritance	Student, Lecturer, HoD and Administrator inherit common identity and authentication fields from abstract User.
Department–Course	One department has 0..* courses; each course belongs to exactly one department.
Course–Assignment	One course has 0..* assignments; each assignment belongs to one course.
Student–Course	Many-to-many resolved through Enrolment; unique student/course pair.
Assignment–Submission	One assignment has 0..* submissions; each submission belongs to one assignment.

Student–Submission	One student has 0..* submissions; each submission belongs to one student.
Submission–MarkRecord	A submission has 0..1 current mark; score revisions are retained in MarkAudit.
MarkRecord–MarkAudit	One mark has 0..* audit entries.
MarkRecord–Moderation	A mark may have multiple decisions if returned and resubmitted.

3.2 Design patterns

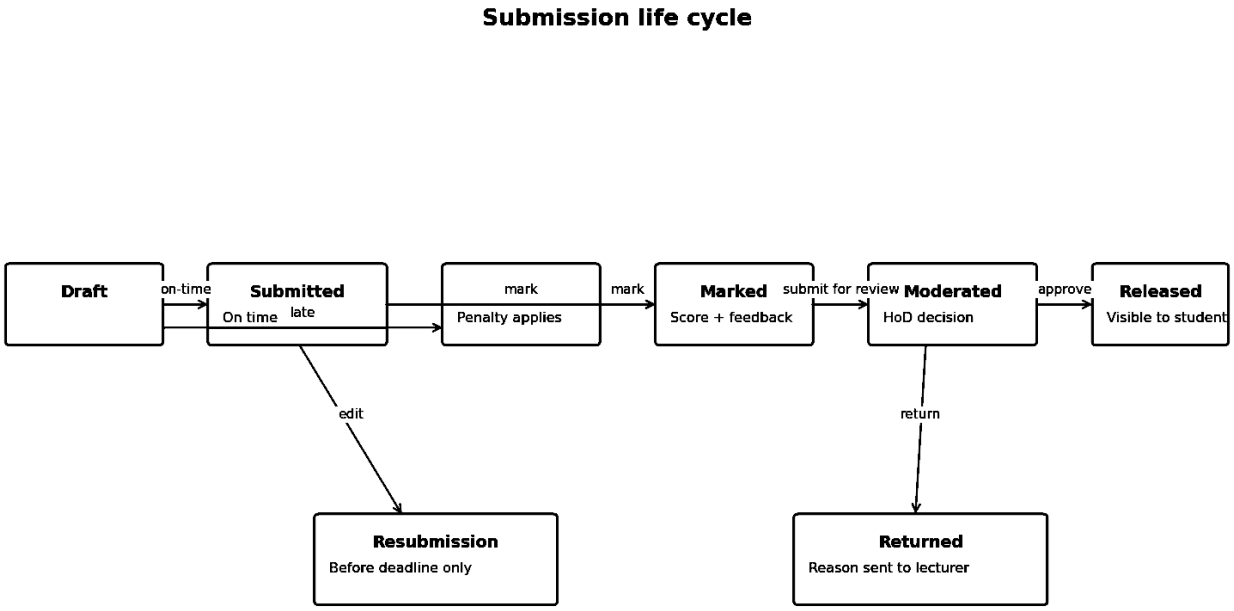
Pattern	Use in this system	Reason
Strategy	LatePenaltyStrategy with StandardLatePenalty.	Encapsulates the late policy so changes do not require rewriting submission workflow code.
Repository / DAO	IRepository<T>, SubmissionRepository, MarkRepository and CourseRepository.	Centralises SQL, parameter binding and row mapping; domain classes do not depend on SQL.
Role polymorphism	Abstract User with four concrete roles.	Shares identity behaviour while keeping role-specific permissions explicit.

3.3 SOLID principles

- Single Responsibility: SubmissionService coordinates submission; StandardLatePenalty calculates penalties; repositories persist data; AuditRepository records audit events.
- Open/Closed: a new penalty strategy can be added without changing the submission workflow.
- Liskov Substitution: each role subtype honours the authentication and permission contract of User.
- Interface Segregation: repository and policy interfaces expose focused operations.
- Dependency Inversion: services depend on repository and policy abstractions rather than concrete SQLite code.

4. Behavioural models

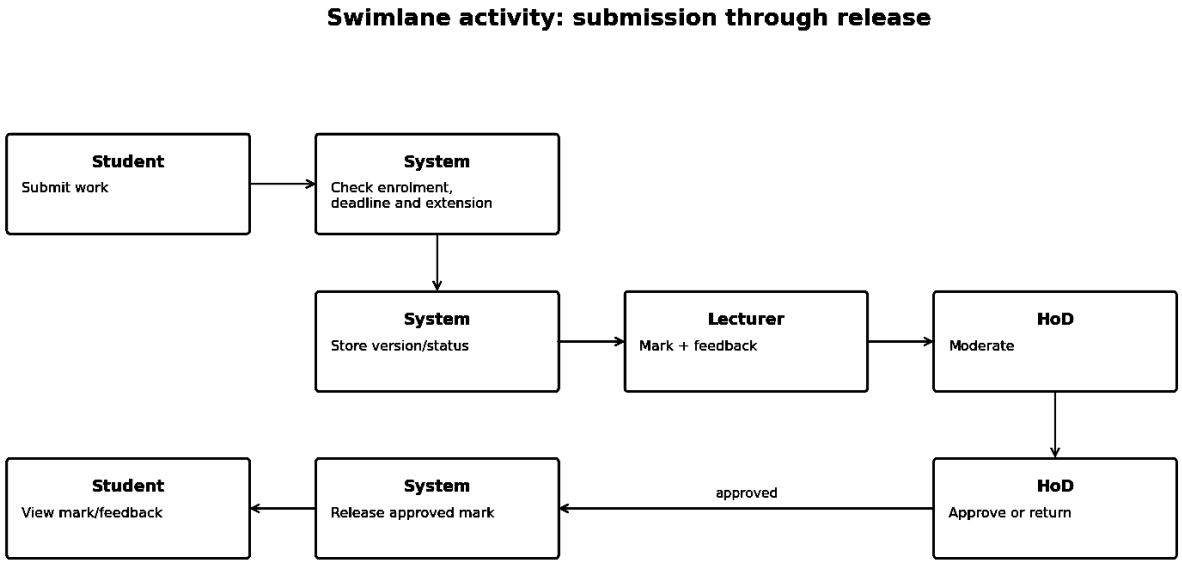
4.1 Submission life-cycle state machine



Submission states and key transitions.

A submission begins as Draft, then becomes Submitted or Late. The lecturer marks it and supplies feedback; it enters moderation. The HoD approves it for release or returns it with a reason. The student cannot view a mark until it is Released. Submission status and mark-release status are related but distinct, so the service layer must validate each transition.

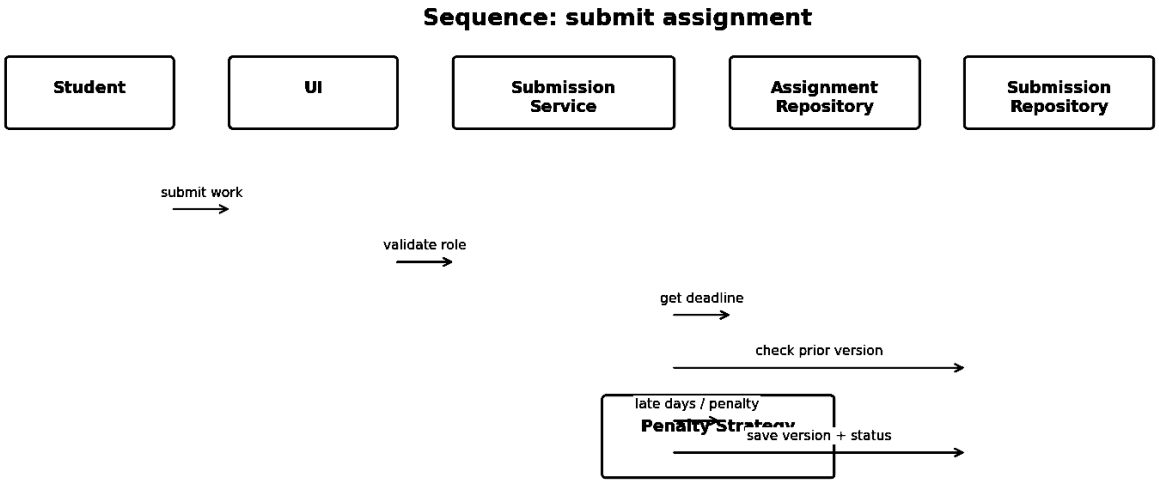
4.2 Activity diagram: submission to release



Swimlane activity model.

The system checks enrolment and deadline, stores the submission and status, and then makes it available for marking. The lecturer records a score and feedback. The HoD reviews departmental marks and either approves or returns them. Only approval permits release; a returned mark loops back to the lecturer.

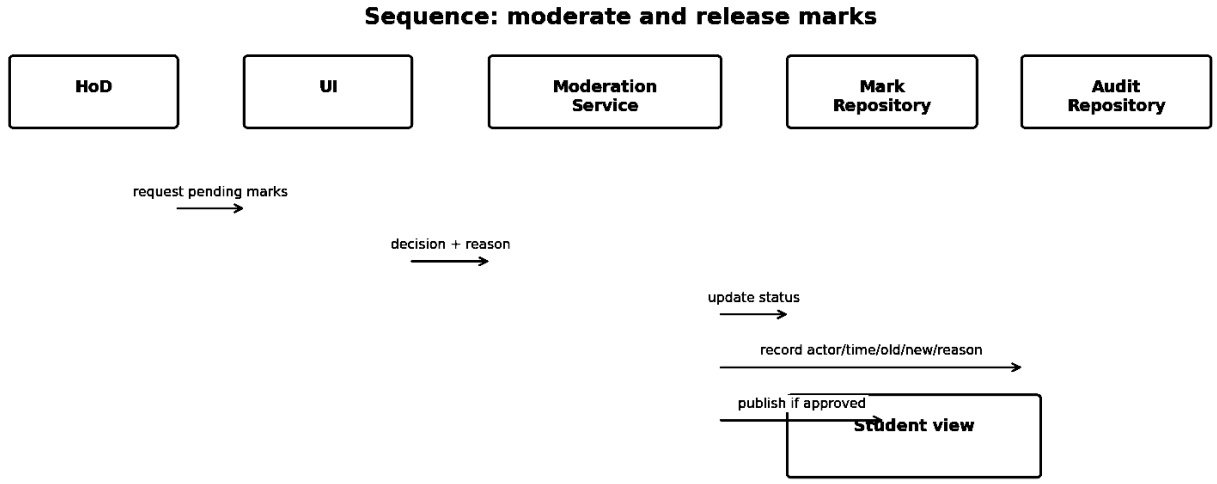
4.3 Sequence: Submit assignment



Submission interaction.

The service validates the student's role and enrolment, retrieves the deadline and any individual extension, and checks for an earlier version. A resubmission is allowed only while the applicable deadline permits it. The penalty strategy classifies late work. The repository saves the new version and status in a transaction. Failed validation must not leave a partial record.

4.4 Sequence: Moderate and release marks



Moderation and release interaction.

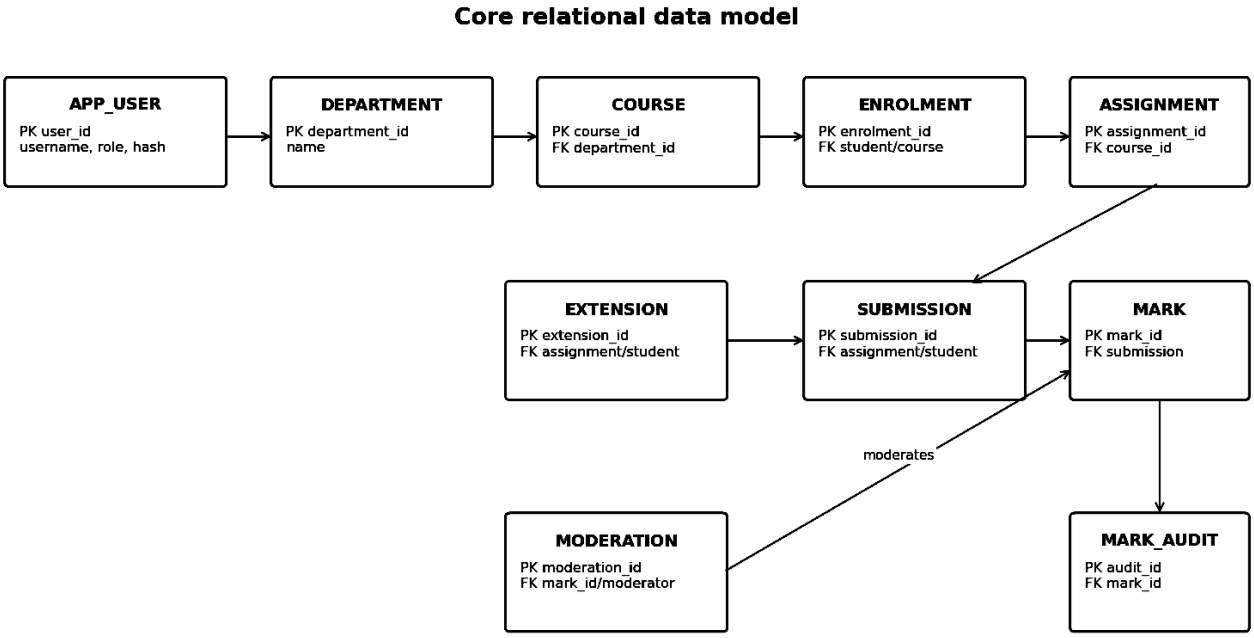
The HoD requests pending marks for their department. The service checks departmental authority before accepting an approval or return decision. It updates the mark status and inserts an audit record in one transaction. Approved marks become visible; returned marks stay hidden and carry the HoD's reason.

4.5 Exception paths

Exception	Expected response
Invalid login / inactive account	Reject access; do not disclose whether a username exists; never log passwords.
Student not enrolled	Reject submission without creating a row.
Work after three days	Record the submission and zero-mark policy outcome; the brief says zero, not rejection.
Score outside 0..maximum	Reject and leave the previous valid mark unchanged.
HoD accesses another department	Deny action and record the access-control event.
Database error during mark change	Rollback mark and audit writes together.

5. Database design

5.1 ERD and normalisation



Core relational entities; the SQL schema defines authoritative keys and constraints.

The schema separates users, departments, courses, enrolments, assignments, submissions, extensions, marks, moderation decisions and audit entries. ENROLMENT resolves the many-to-many student/course relationship. Department names are stored once in DEPARTMENT, and mark history is separate from the current mark.

5.2 Third Normal Form

- 1NF: each column contains one value; courses, marks and feedback are not stored as comma-separated lists.
- 2NF: ENROLMENT has a single key and a uniqueness constraint on student_id/course_id; non-key data depends on the enrolment record.
- 3NF: department names are not repeated in COURSE; assignment and submission details are stored in their own tables; moderation and audit history are separate.

5.3 Data dictionary

Table	Key fields and constraints	Purpose
APP_USER	user_id PK; username UNIQUE; password_hash NOT NULL; role CHECK	Identity, login hash and role.
DEPARTMENT	department_id PK; name UNIQUE	School department.
COURSE	course_id PK; department_id FK; code UNIQUE	Course belongs to a department.
ENROLMENT	enrolment_id PK; student_id/course_id FKs; pair UNIQUE	Student-course registration.
ASSIGNMENT	assignment_id PK; course_id FK; created_by FK; max_mark > 0	Assignment, deadline and maximum mark.
EXTENSION	extension_id PK; assignment/student/grantor FKs	Individual deadline extension and reason.
SUBMISSION	submission_id PK; assignment/student FKs; version >= 1; status CHECK	Submitted content and version history.
MARK	mark_id PK; submission_id UNIQUE; score >= 0; feedback; status CHECK	Current score, feedback and release state.
MODERATION	moderation_id PK; mark/moderator FKs; decision CHECK	Approval/return history.
MARK_AUDIT	audit_id PK; mark/actor FKs; old/new score; reason	Append-only mark-change history.

5.4 SQLite schema

```
PRAGMA foreign_keys = ON;

CREATE TABLE department (
  department_id INTEGER PRIMARY KEY,
  name TEXT NOT NULL UNIQUE
);

CREATE TABLE app_user (
  user_id INTEGER PRIMARY KEY,
  username TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  full_name TEXT NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('STUDENT','LECTURER','HOD','ADMIN')),
  department_id INTEGER,
  active INTEGER NOT NULL DEFAULT 1 CHECK (active IN (0,1)),
  FOREIGN KEY (department_id) REFERENCES department(department_id)
);

CREATE TABLE course (
  course_id INTEGER PRIMARY KEY,
  department_id INTEGER NOT NULL,
  code TEXT NOT NULL UNIQUE,
  title TEXT NOT NULL,
  FOREIGN KEY (department_id) REFERENCES department(department_id)
);

CREATE TABLE enrolment (
  enrolment_id INTEGER PRIMARY KEY,
  student_id INTEGER NOT NULL,
  course_id INTEGER NOT NULL,
  enrolled_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(student_id, course_id),
  FOREIGN KEY (student_id) REFERENCES app_user(user_id),
  FOREIGN KEY (course_id) REFERENCES course(course_id)
);

CREATE TABLE assignment (
  assignment_id INTEGER PRIMARY KEY,
  course_id INTEGER NOT NULL,
  created_by INTEGER NOT NULL,
  title TEXT NOT NULL,
  description TEXT,
  deadline TEXT NOT NULL,
  max_mark REAL NOT NULL CHECK(max_mark > 0),
  FOREIGN KEY (course_id) REFERENCES course(course_id),
  FOREIGN KEY (created_by) REFERENCES app_user(user_id)
);

CREATE TABLE extension (
  extension_id INTEGER PRIMARY KEY,
  assignment_id INTEGER NOT NULL,
  student_id INTEGER NOT NULL,
  new_deadline TEXT NOT NULL,
  granted_by INTEGER NOT NULL,
  reason TEXT NOT NULL,
  granted_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (assignment_id) REFERENCES assignment(assignment_id),
  FOREIGN KEY (student_id) REFERENCES app_user(user_id),
  FOREIGN KEY (granted_by) REFERENCES app_user(user_id)
);
```

```
CREATE TABLE submission (
  submission_id INTEGER PRIMARY KEY,
  assignment_id INTEGER NOT NULL,
  student_id INTEGER NOT NULL,
  version INTEGER NOT NULL CHECK(version >= 1),
  submitted_at TEXT NOT NULL,
  status TEXT NOT NULL CHECK(status IN ('SUBMITTED','LATE','MARKED','MODERATED','RELEASED')),
  content_text TEXT NOT NULL,
  UNIQUE(assignment_id, student_id, version),
  FOREIGN KEY (assignment_id) REFERENCES assignment(assignment_id),
  FOREIGN KEY (student_id) REFERENCES app_user(user_id)
);

CREATE TABLE mark (
  mark_id INTEGER PRIMARY KEY,
  submission_id INTEGER NOT NULL UNIQUE,
  score REAL NOT NULL CHECK(score >= 0),
  feedback TEXT NOT NULL,
  marked_by INTEGER NOT NULL,
  status TEXT NOT NULL CHECK(status IN ('DRAFT','PENDING_MODERATION','RETURNED','APPROVED','RELEASED')),
  released_at TEXT,
  FOREIGN KEY (submission_id) REFERENCES submission(submission_id),
  FOREIGN KEY (marked_by) REFERENCES app_user(user_id)
);

CREATE TABLE moderation (
  moderation_id INTEGER PRIMARY KEY,
  mark_id INTEGER NOT NULL,
  moderator_id INTEGER NOT NULL,
  decision TEXT NOT NULL CHECK(decision IN ('APPROVE','RETURN')),
  reason TEXT NOT NULL,
  decided_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (mark_id) REFERENCES mark(mark_id),
  FOREIGN KEY (moderator_id) REFERENCES app_user(user_id)
);

CREATE TABLE mark_audit (
  audit_id INTEGER PRIMARY KEY,
  mark_id INTEGER NOT NULL,
  changed_by INTEGER NOT NULL,
  changed_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  old_score REAL,
  new_score REAL NOT NULL,
  reason TEXT NOT NULL,
  FOREIGN KEY (mark_id) REFERENCES mark(mark_id),
  FOREIGN KEY (changed_by) REFERENCES app_user(user_id)
);

CREATE INDEX idx_assignment_course ON assignment(course_id);
CREATE INDEX idx_submission_assignment_student ON submission(assignment_id, student_id);
CREATE INDEX idx_mark_status ON mark(status);
```

5.5 Schema and implementation notes

- Enable PRAGMA foreign_keys = ON on every SQLite connection.
- The service must check score <= assignment.max_mark because that rule spans tables.
- Only a lecturer assigned to the course may grant an extension; the service checks this and records the reason.
- Mark updates and MARK_AUDIT inserts must share one transaction.
- The schema uses a single APP_USER table with a role field to avoid duplicating authentication fields.

5.6 Class-to-table mapping

Class / concept	Database mapping	Reason

User and role subclasses	APP_USER role column	Single-table inheritance keeps shared login fields in one place.
Department / Course	DEPARTMENT / COURSE	Independent entities connected by foreign key.
Student-course relationship	ENROLMENT	Associative entity implements many-to-many relationship.
Assignment / Submission	ASSIGNMENT / SUBMISSION	Each resubmission is a separate version row.
Extension	EXTENSION	Preserves student, grantor and reason.
MarkRecord	MARK	Stores current score, feedback and release state.
Moderation	MODERATION	Preserves each HoD decision.
MarkAudit	MARK_AUDIT	Separates immutable history from current score.

6. Requirements traceability

The Phase 2 SRS was not supplied with this brief, so the identifiers below are provisional and should be aligned with the approved SRS before Gate 2.

Provisional requirement	Use case	Design elements	Tables
FR-01 Login and role access	Log in	User hierarchy; AuthenticationService	APP_USER
FR-02 Admin manages users/departments/courses	Manage accounts/courses	Administrator; CourseService	APP_USER, DEPARTMENT, COURSE
FR-03 Admin enrols students	Enrol student	Administrator; EnrolmentService	ENROLMENT
FR-04 Lecturer creates assignment	Create assignment	Lecturer; Assignment	ASSIGNMENT
FR-05 Student submits/resubmits	Submit assignment	SubmissionService; state model	SUBMISSION
FR-06 Individual extensions	Grant extension	Assignment/Extension; permission check	EXTENSION
FR-07 Late penalty	Calculate penalty	LatePenaltyStrategy	SUBMISSION, MARK
FR-08 Lecturer marks and gives feedback	Mark submission	MarkingService; MarkRecord	MARK
FR-09 HoD moderates marks	Moderate marks	HeadOfDepartment; ModerationService	MODERATION
FR-10 Only released marks visible	View marks	Student; release-state guard	MARK
FR-11 Audit every mark change	Update mark	MarkAudit; AuditRepository	MARK_AUDIT
NFR-01 Offline/local use	All use cases	C++ app + local SQLite	All local tables

7. Design decisions and open points

7.1 Security and integrity

- Store salted password hashes using a recognised password-hashing library; never store plaintext passwords.
- Check role and resource ownership in services, not only by hiding menu options.
- Use parameterised SQL and validate input; catch database errors without crashing.
- Use transactions for mark changes, moderation updates and audit inserts.
- Student queries must filter for RELEASED marks only.

7.2 Items to confirm before implementation

Issue	Current design decision	Confirmation needed
Late-day boundary	Each started 24-hour period counts as one day.	Confirm calendar-day versus 24-hour interpretation.
After three days	Store attempt and award zero rather than reject.	Confirm whether the school wants the submission retained.
Resubmission history	Each version stored separately.	Confirm whether lecturers compare versions.
Released mark correction	Any correction after release should be re-moderated.	Confirm exact approval workflow.
Role-specific data	One APP_USER table with optional department_id.	Add role-specific tables if the SRS requires them.

7.3 Conclusion

The design separates interface, business rules and persistence, models the four roles, and controls the path from submission to moderation and release. Strategy and Repository/DAO patterns isolate late-policy calculations and database access. The schema supports enrolment, deadlines, extensions, submission versions, feedback, moderation and audit history. The design should be checked against the approved Phase 2 SRS before implementation, especially the late-day boundary and treatment of work submitted after three days.

References

National University of Lesotho, Department of Mathematics and Computer Science. (2026). CS4433 – Software Engineering: Individual Assignment, case study and Phase 3 system design requirements.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

SQLite. (n.d.). SQLite documentation. https://www.sqlite.org/docs.html